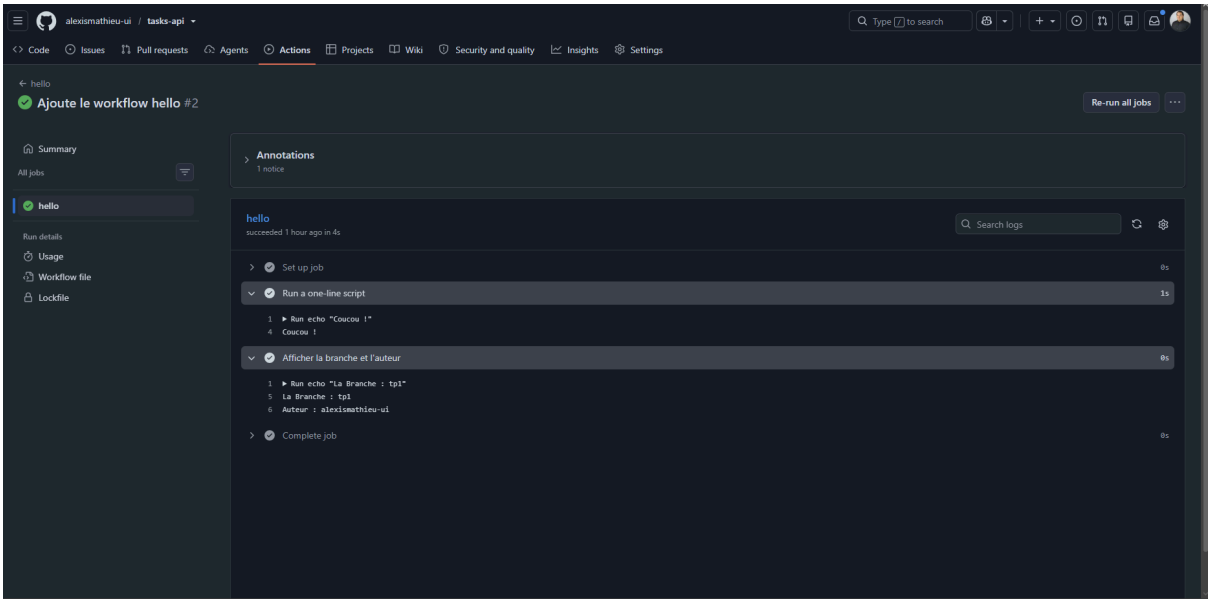
Dossier TP

Mathieu
Alexis
02/11 - 05/11
Lien tasks-api github : https://github.com/alexismathieu-ui/tasks-api
Lien tasks-config github : https://github.com/alexismathieu-ui/tasks-config
(Les screens sont aussi disponible dans le tasks-api)

TP1 :

Screen 1 :



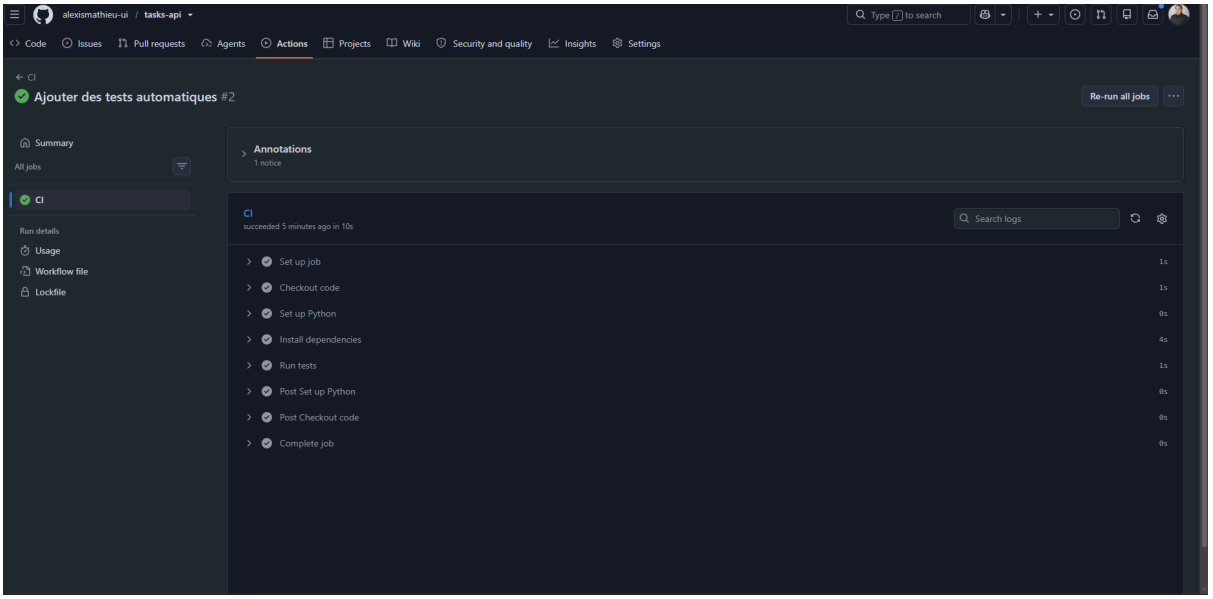
Légende : Workflow Hello a bien été exécuté

Question : Combien de temps a duré le run ? Où voit-on les logs de chaque étape ?

Le run a duré 4 secondes pour le hello.yml mais en total sa a duré 7 secondes. On voit les logs de chaque étape du workflow en cliquant sur notre workflow ce qui va nous afficher chaque étape des logs avec le résultat.

Partie 2 :

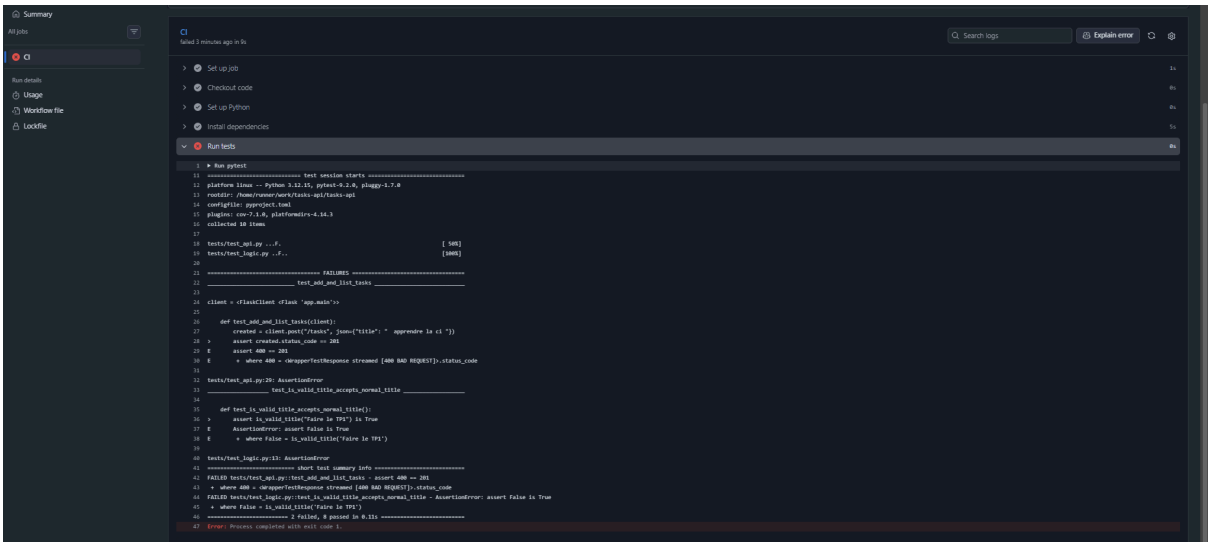
Screen 1 :



Légende : Workflow CI a bien été exécuté et on voit chaque étape qui est bien valider.

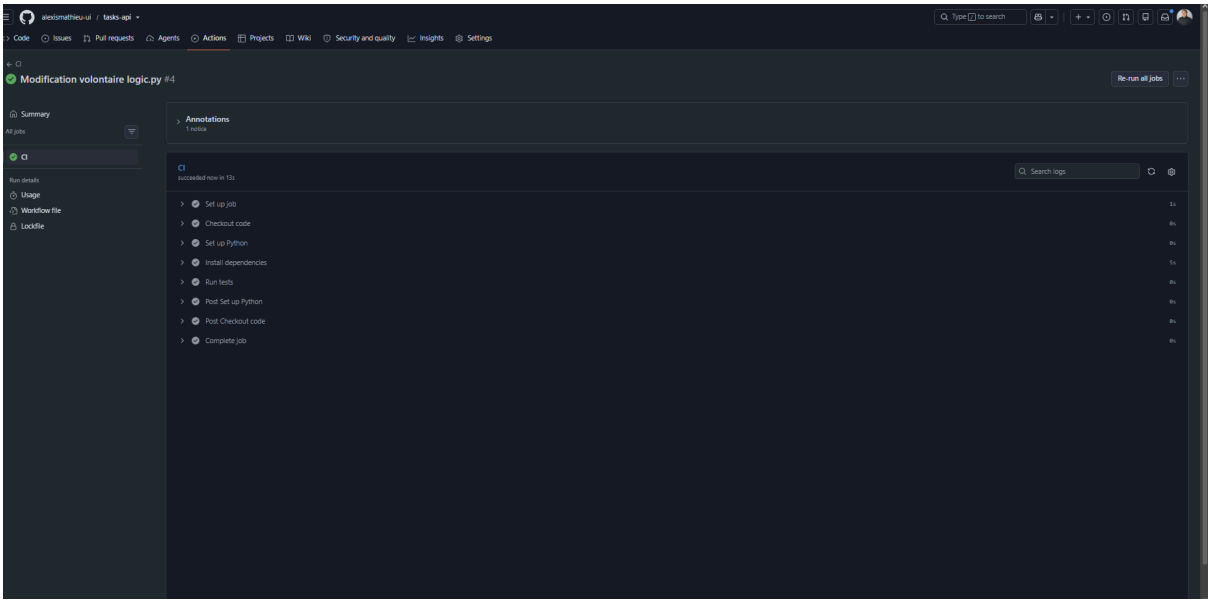
Partie 3 :

Screen 1 :



Légende : Workflow CI qui est rouge car il a ratée au test pytest.

Screen 2 :



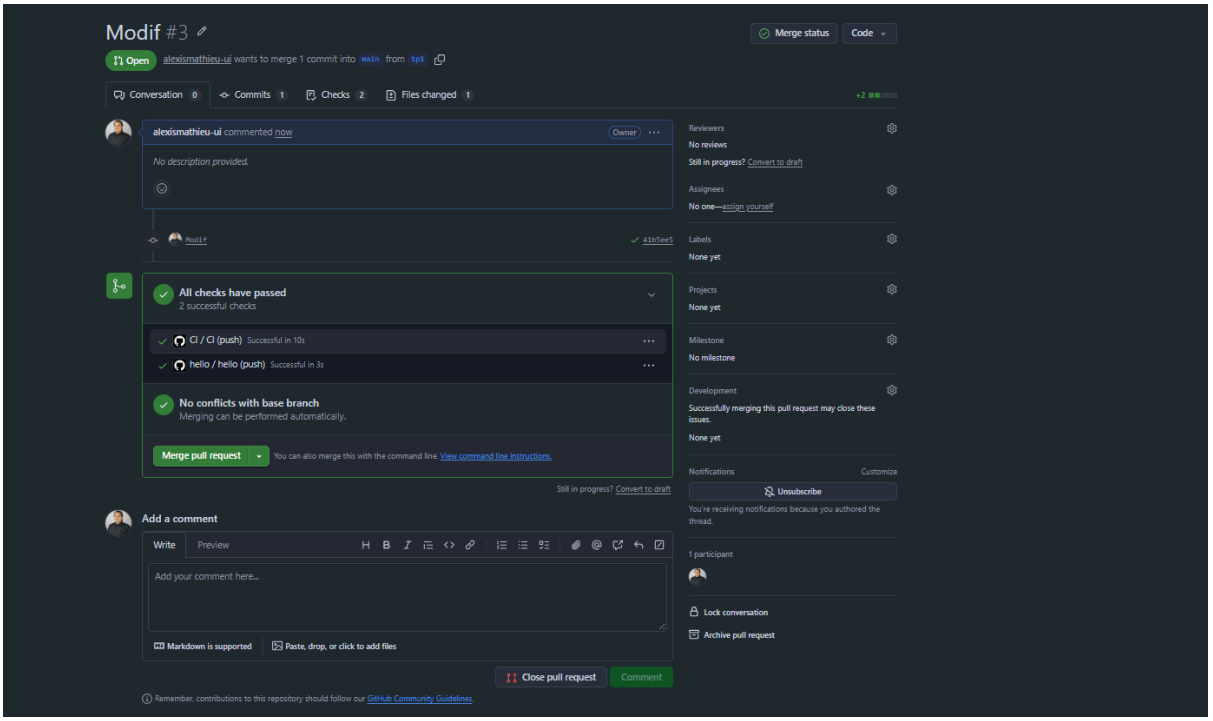
Légende : Workflow CI qui est vert, toutes les étapes ont bien été exécuté et que le problème a été corrigé.

Question : Quel test a échoué ? À quelle ligne ? Qu'est-ce qui a permis de le trouver rapidement ?

Deux tests ont échoué : - test_is_valid_title_accepts_normal_title dans tests/test_logic.py à la ligne 13 et - test_add_and_list_tasks dans tests/test_api.py à la ligne 29. On les repère immédiatement car GitHub Actions déplie automatiquement l'étape Run tests en échec, et le rapport que fait pytest grâce au chevron sur la ligne fautive, le format fichier.py:ligne, ainsi que le résumé short test summary info en fin de log.

Partie 4 :

Screen 1 :



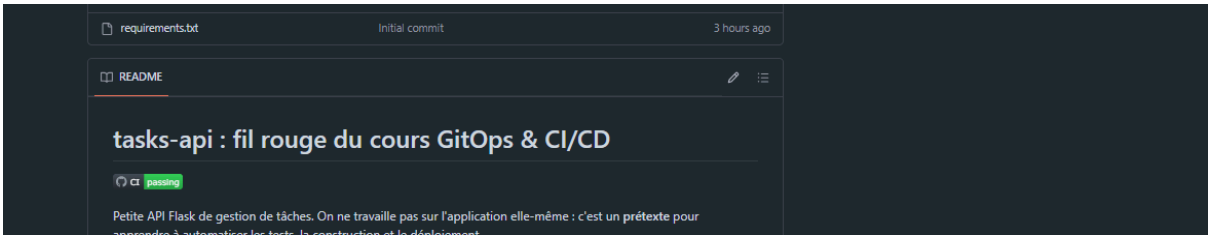
Légende : La pull request avec les checks verts avant la fusion.

Screen 2 :



Légende : La pull request merge.

Screen 3 :



Légende : Le README.md avec le badge affiché

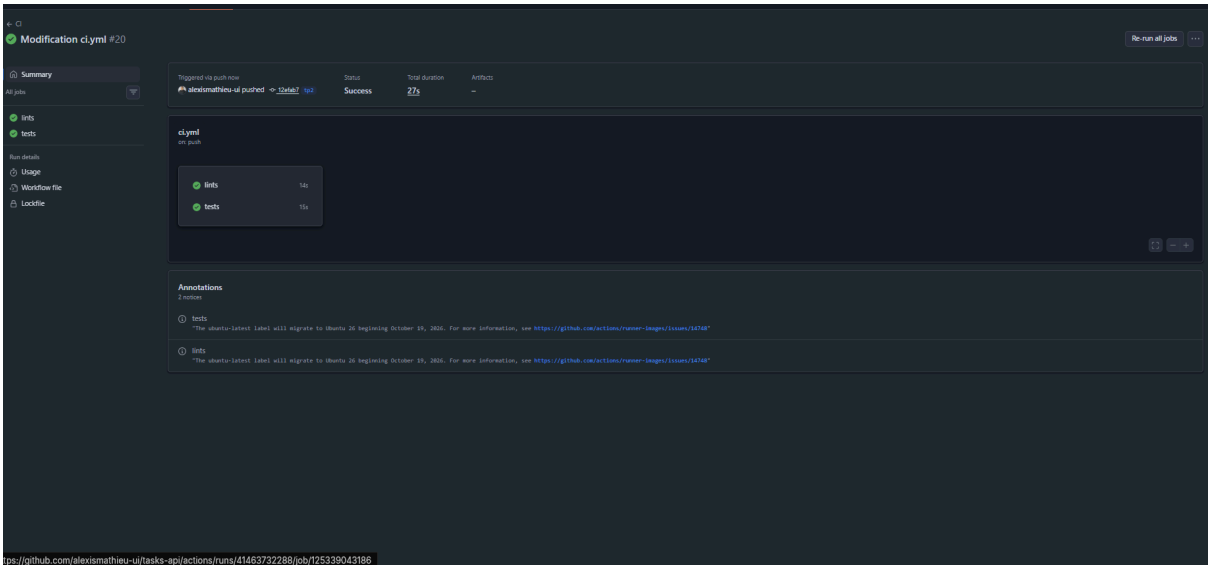
Question : Pourquoi attend-on que la CI soit verte avant de fusionner ?

Pour garantir que la branche main reste en permanence stable, fonctionnelle et déployable. Si la CI est rouge, cela signifie que les modifications introduisent une erreur aux tests. Bloquer la fusion empêche de propager un bug à toute l'équipe ou en production.

TP2 :

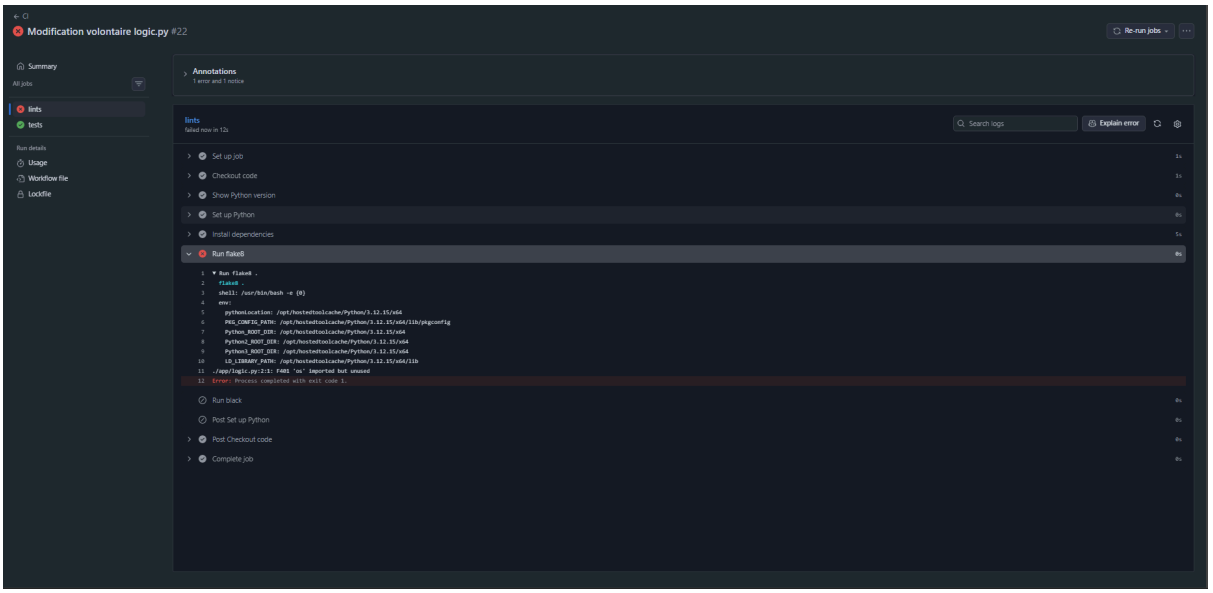
Partie 1 :

Screen 1 :



Légende : Lint et le test qui a bien été effectuer

Screen 2:



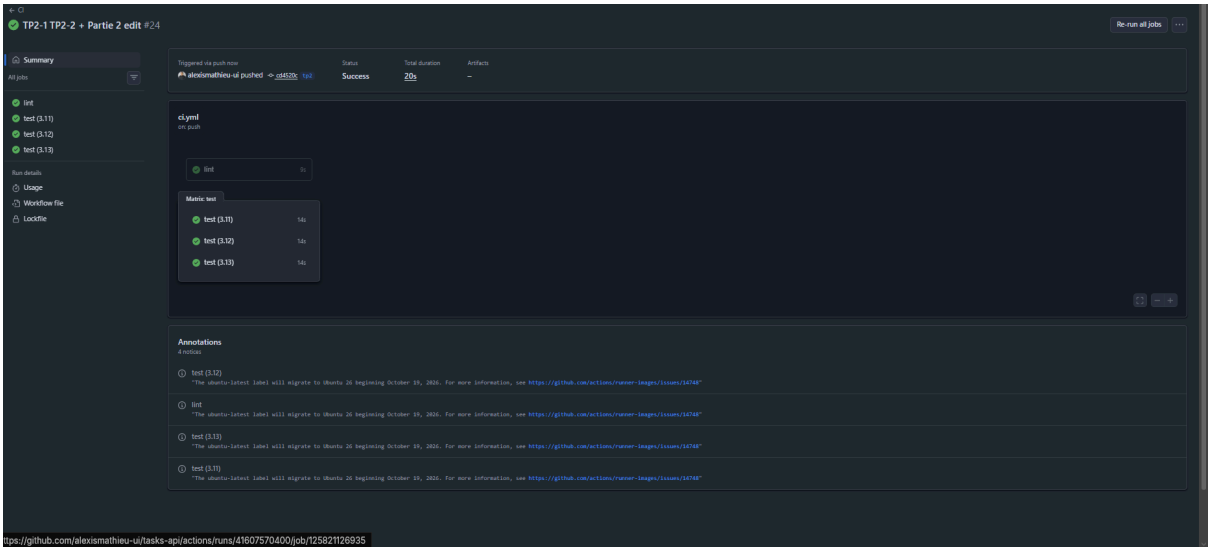
Légende : Lint a une erreur au niveau du flake8

Question : Quelle différence entre un linter (flake8) et un formateur (black) ? Pourquoi séparer `lint` et `test` en deux jobs ?

Le linter (flake8) analyse le code pour détecter des erreurs de syntaxe, des bugs potentiels, des variables/imports inutilisés ou du code suspect.Le formateur (black) s'occupe uniquement de la présentation visuelle et du style pour que tout le code de l'équipe soit uniforme.Séparer les jobs permet que ça aille plus rapidement, ça permet aussi d'avoir un diagnostique rapide

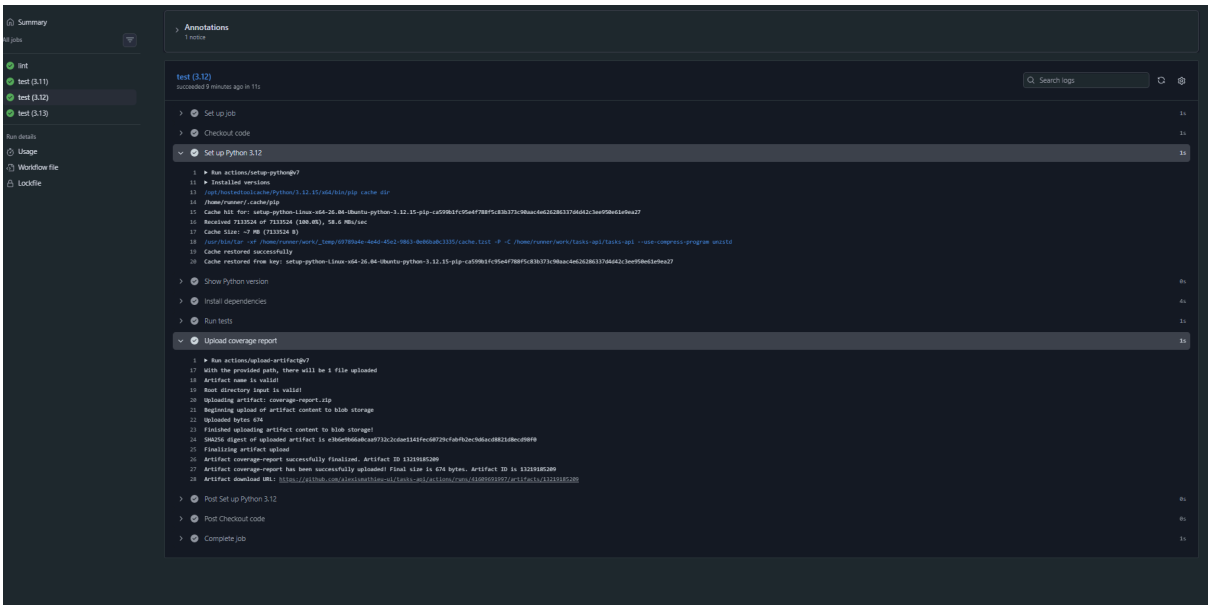
Partie 2 :

Screen 1 :



Légende : Les 3 tests réussi

Screen 2 :



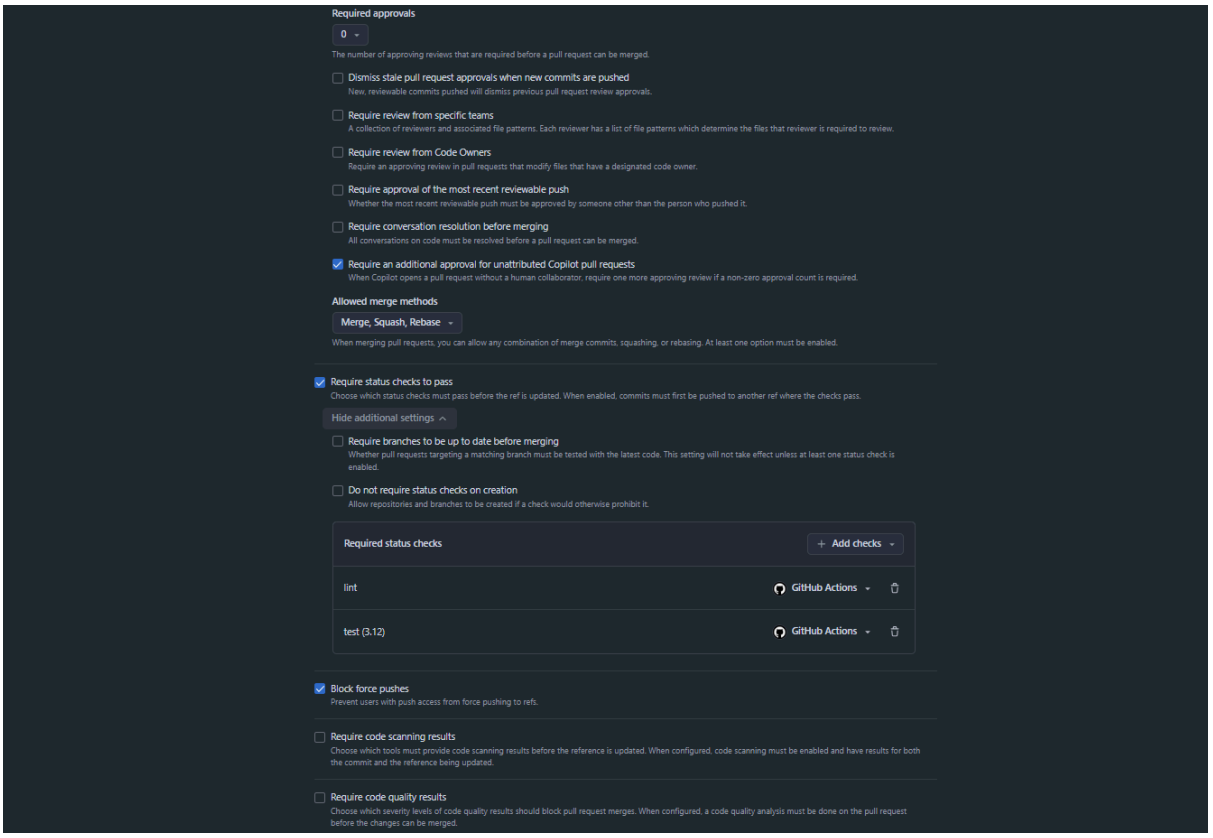
Légende : 2ème run du test 3.12

Question : Pourquoi tester sur plusieurs versions de Python ? Que gagne-t-on avec le cache ? Un artefact, c'est quoi ?

Tester sur plusieurs versions de Python garantit que l'application reste compatible partout. Le cache pip évite de retélécharger les paquets à chaque run pour accélérer la CI, et un artefact est un fichier généré pendant le job que GitHub conserve après la destruction de la machine virtuelle pour qu'on puisse le télécharger.

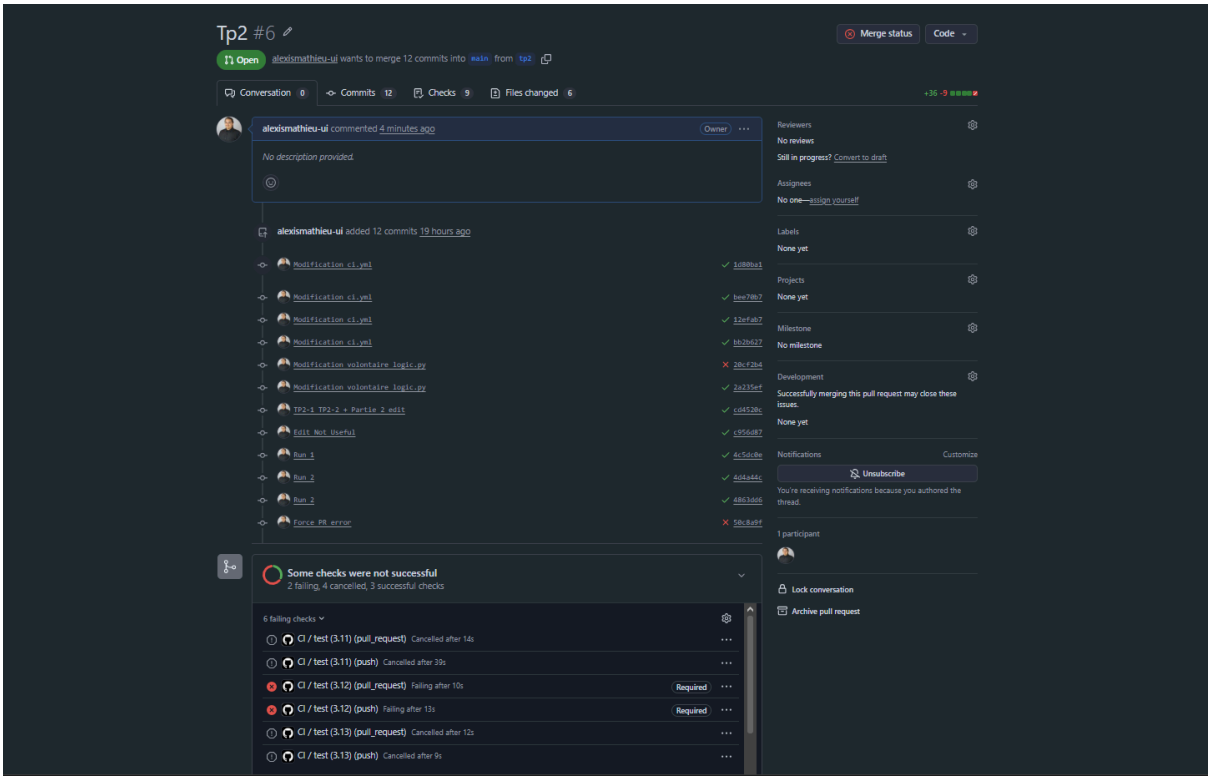
Partie 3 :

Screen 1 :



Légende : Règle de protection et vérification branch main

Screen 2 :



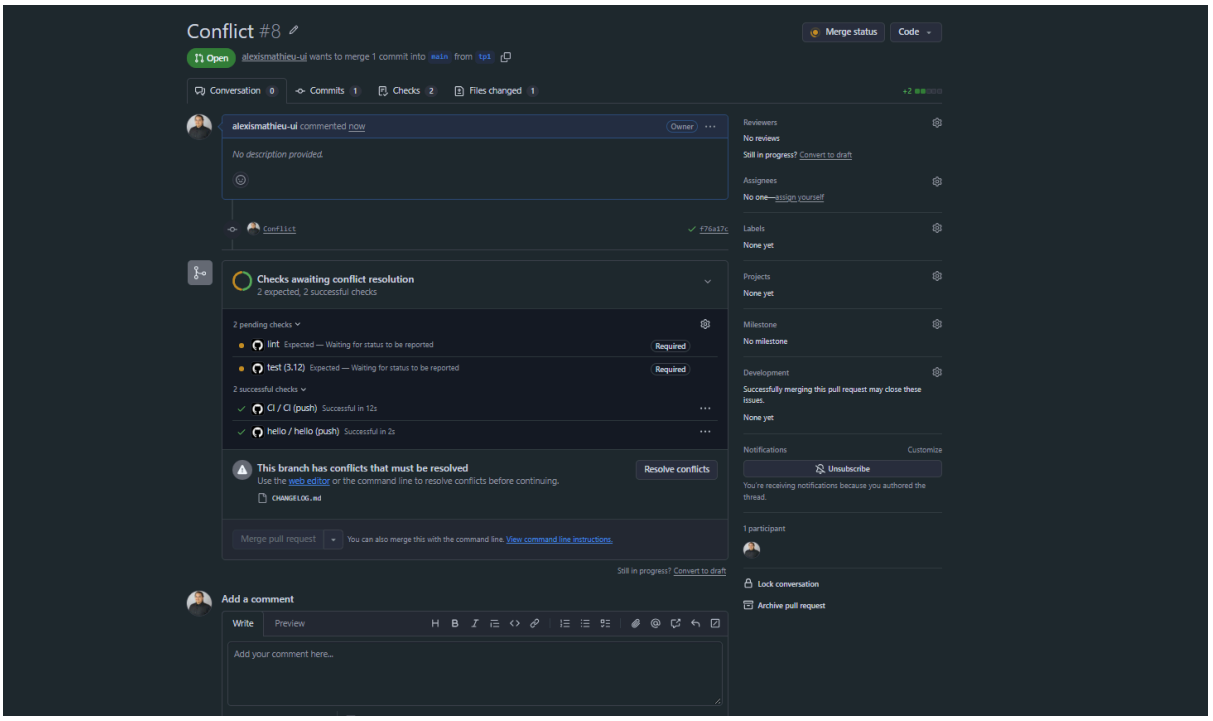
Légende : Fusion bloquer à cause de la PR

Question : Pourquoi interdire de pousser directement sur `main` ?

Interdire de pousser directement sur main oblige chaque modification à passer par une Pull Request et à valider la CI avant fusion, ce qui garantit que la branche principale reste toujours propre et fonctionnelle.

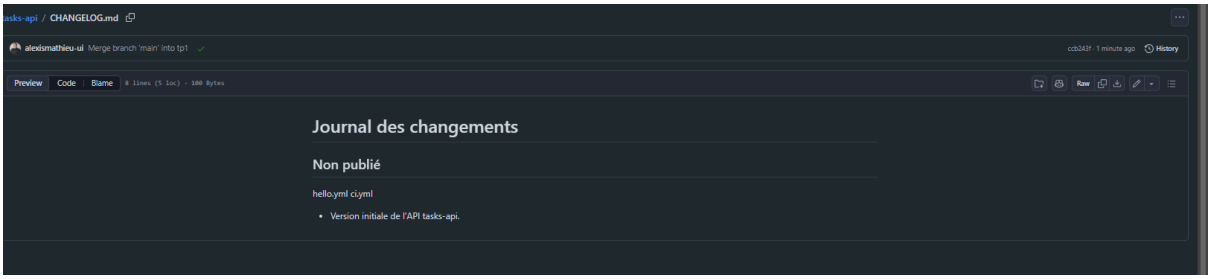
Partie 4 :

Screen 1 :



Légende : Message de conflit PR B

Screen 2 :



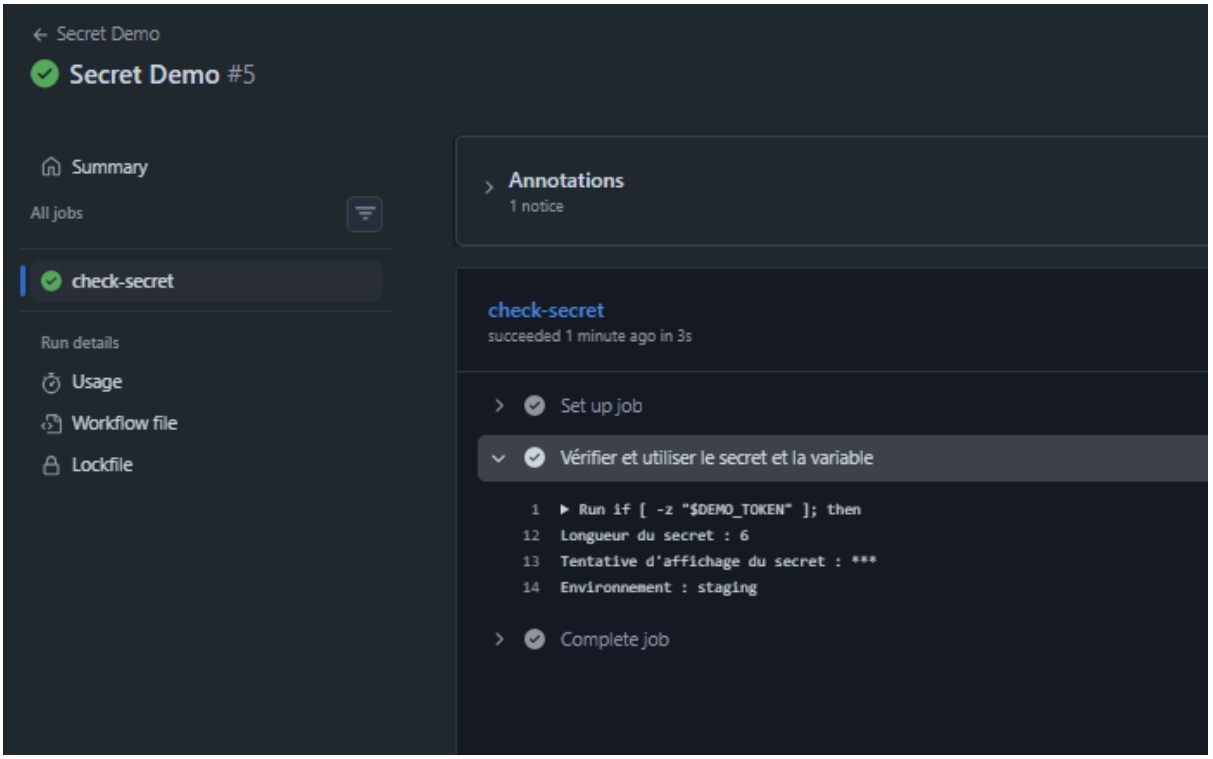
Légende : Changelog.md final

Question : Pourquoi Git ne peut-il pas résoudre ce conflit seul ? Qu'est-ce qui a vérifié le code après la résolution ?

Git ne peut pas résoudre ce conflit seul car les deux branches ont modifié exactement la même ligne de CHANGELOG.md avec deux textes différents. Une fois le conflit résolu et poussé, c'est la CI qui s'est relancée automatiquement sur la PR pour revalider le code avant fusion.

Partie 5

Screen 1 :



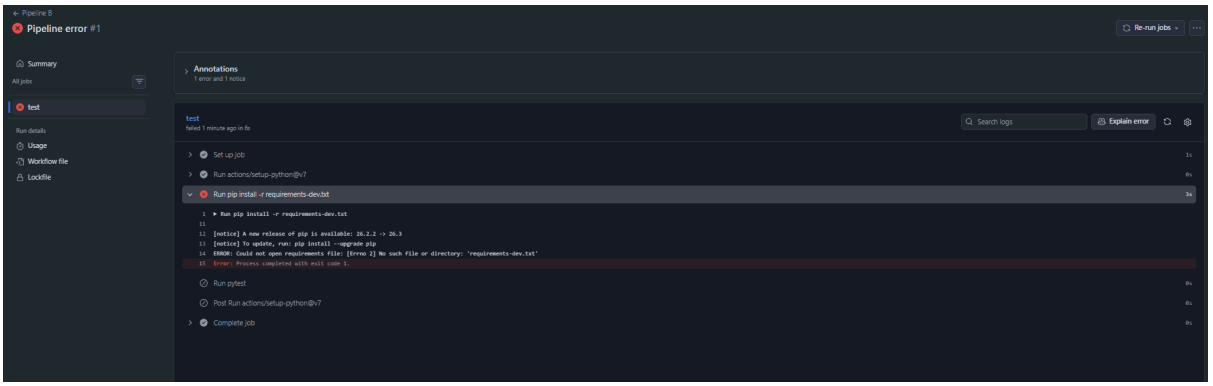
Légende : Valeur secret masquer

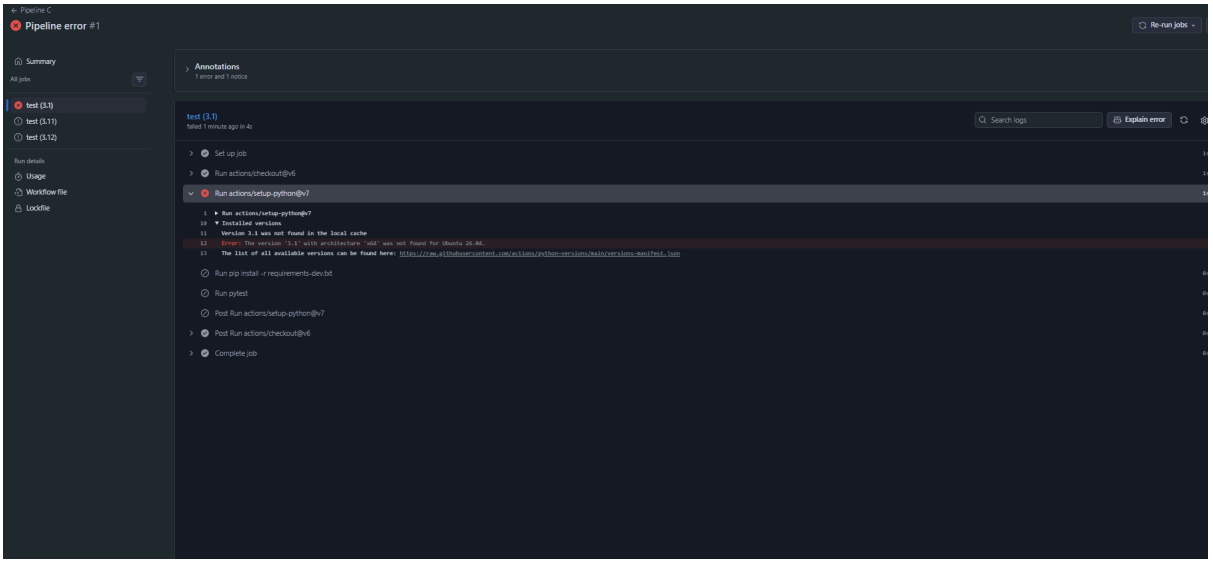
Question : Pourquoi GitHub masque-t-il le secret ? Que se passe-t-il si on le commite dans le dépôt ? Secret ou variable : comment choisir ?

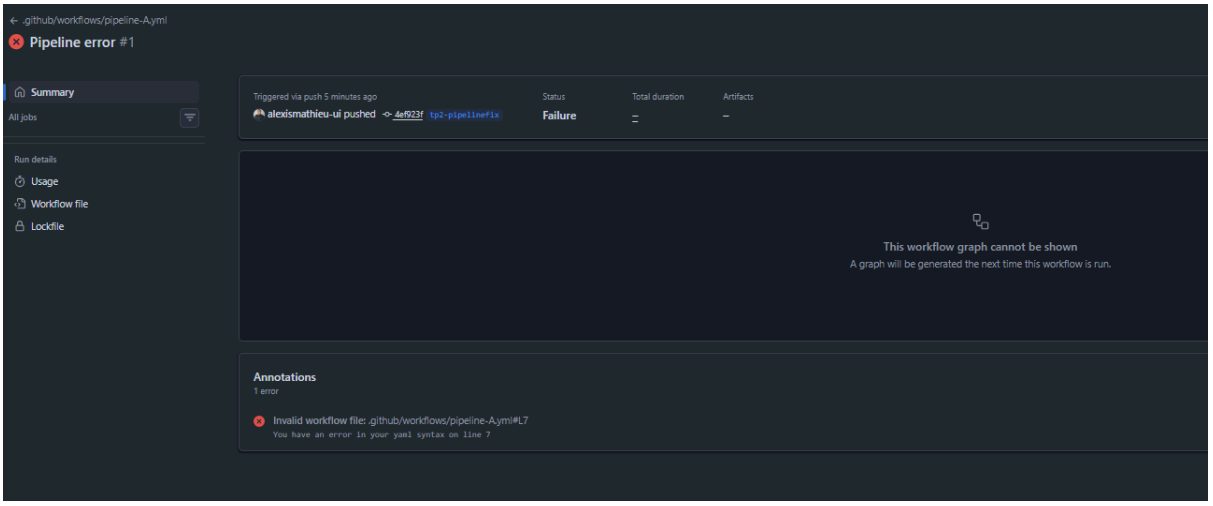
GitHub masque les secrets par *** pour éviter qu'ils ne fuitent en clair dans les logs publics. Si un secret est commité dans Git, il reste dans l'historique et est compromis : on utilise un secret pour les données sensibles et une variable pour la configuration non sensible

Partie 6

Screen 1 - 2 - 3 :

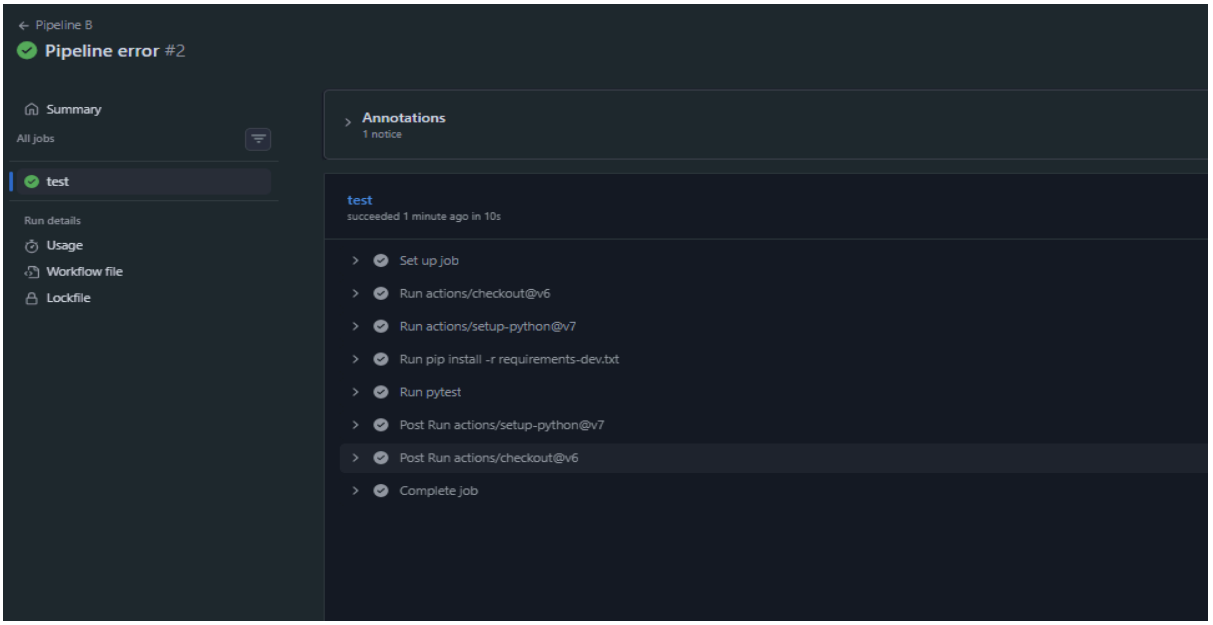


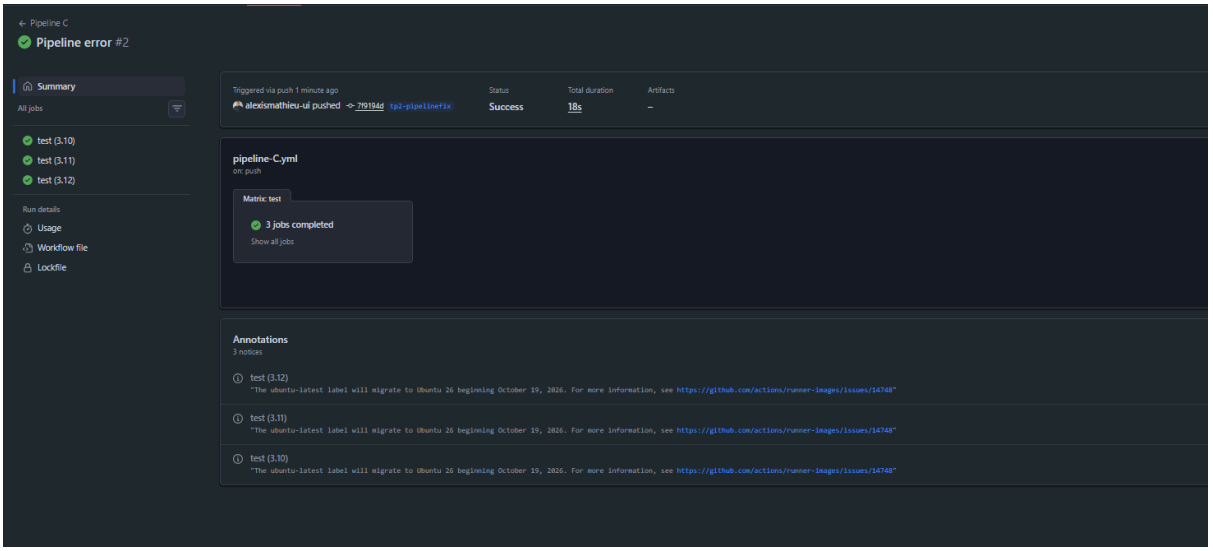


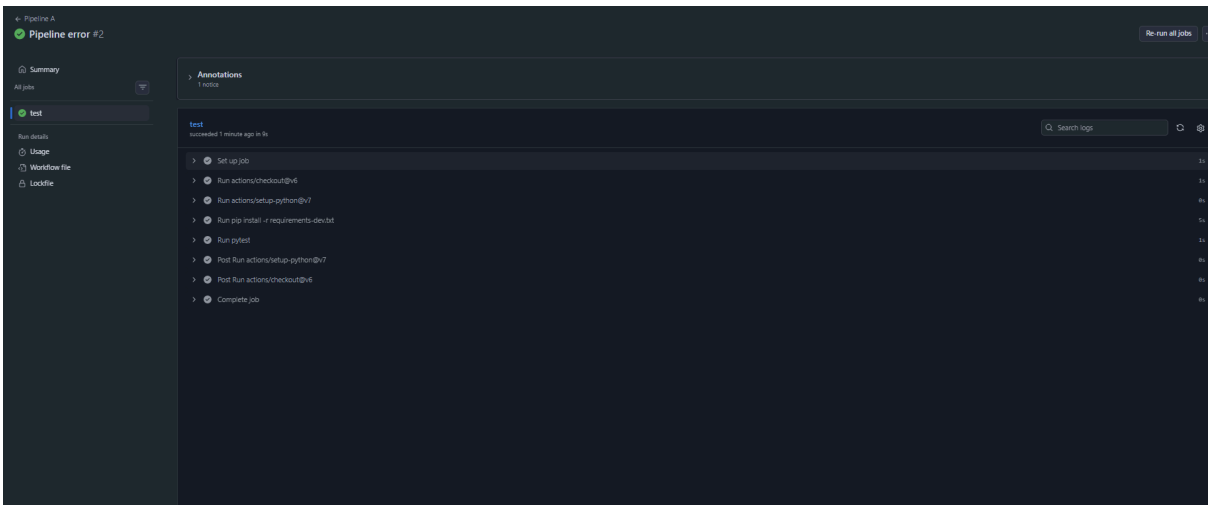


Légende : Pipe A , B et C échec

Screen 4 - 5 - 6 :







Légende : Pipe A , B , C réparer

Question : Pour chaque pipeline : quel était le bug, comment l'avez-vous trouvé, comment l'éviter à l'avenir ?

Pipeline A : Il y avait un espace en trop devant - run: pip install, détecté par l'erreur Invalid workflow file. On l'évite avec un validateur YAML dans l'IDE.

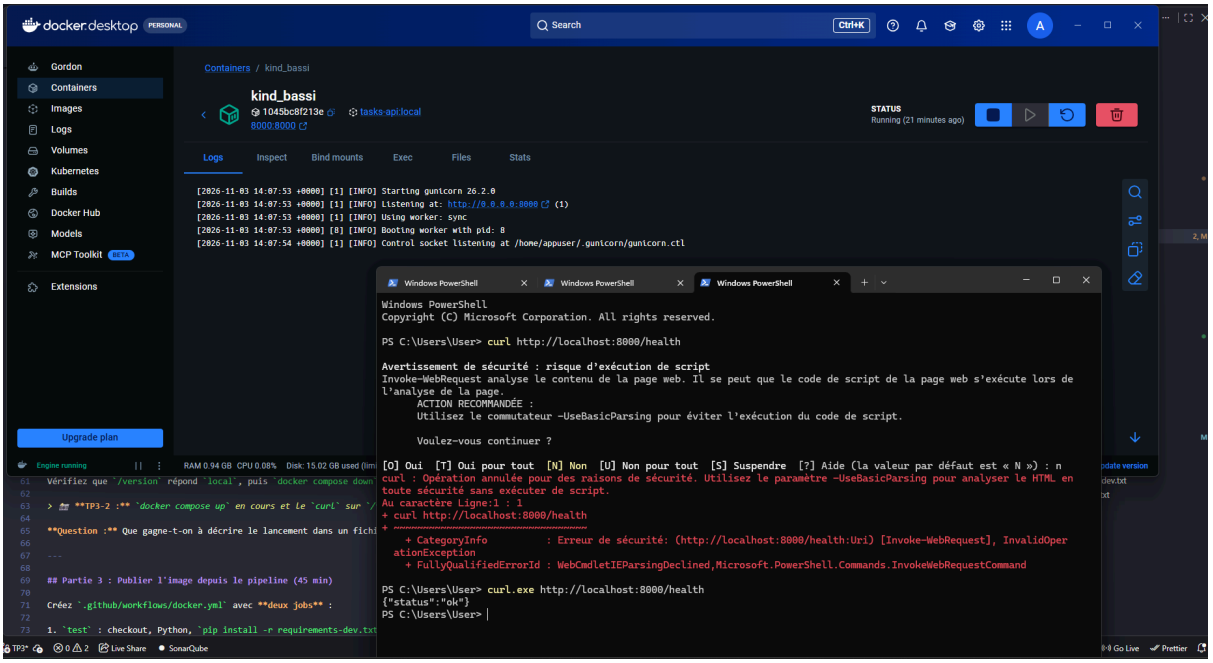
Pipeline B : Il manquait actions/checkout@v6 au début, détecté car requirements-dev.txt était introuvable .On l'évite en clonant toujours le dépôt en première étape.

Pipeline C : La version 3.10 sans guillemets dans la matrice était lue comme le nombre 3.1 par YAML .On l'évite en mettant toujours les versions entre guillemets.

TP3 :

Partie 1

Screen 1 :



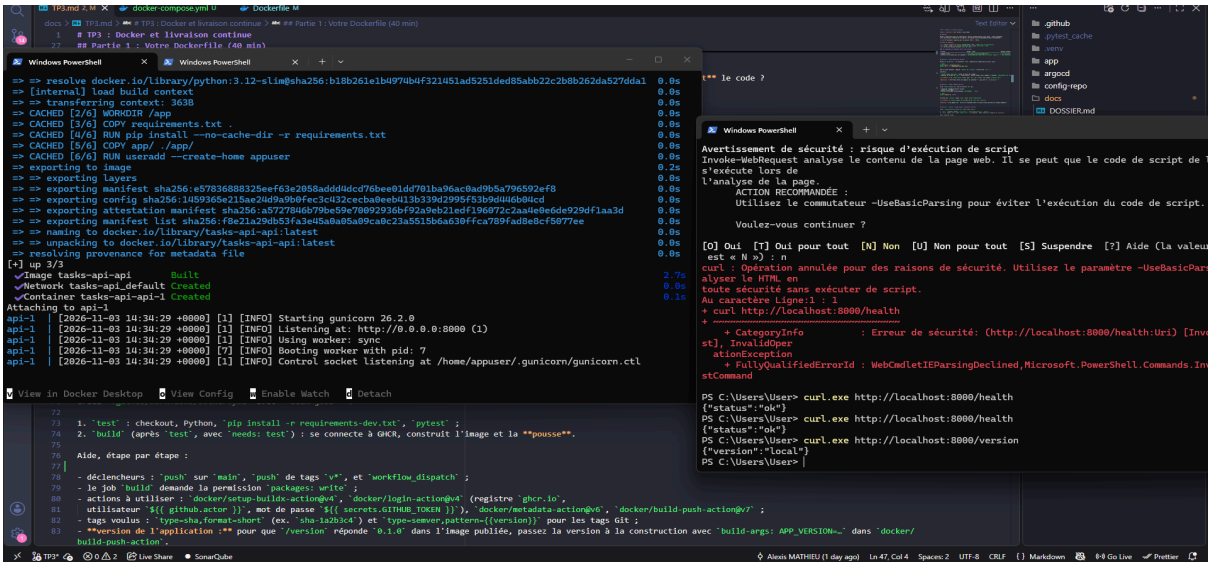
Légende : Docker build réussi + curl “/health” Okay

Question : Différence entre une image et un conteneur ? À quoi sert le `.dockerignore` ?

Une image est un modèle figé en lecture seule tandis qu'un conteneur est l'instance en cours d'exécution lancée depuis l’image.
Le .dockerignore évite de copier les dossiers inutiles ou sensibles pour alléger et sécuriser l'image.

Partie 2

Screen 1 :



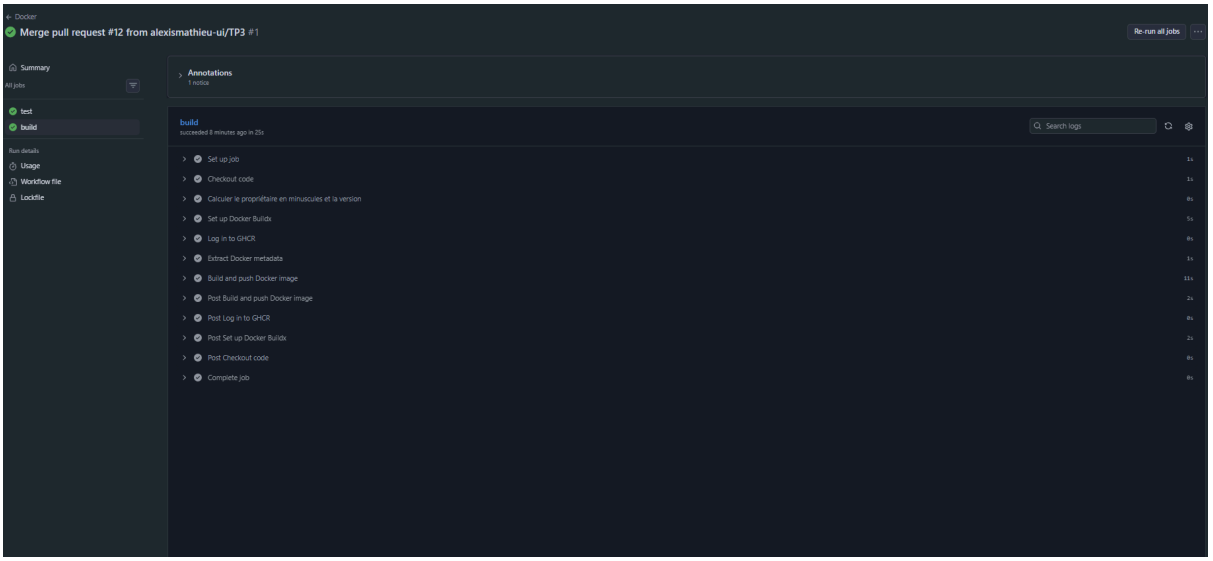
Légende : docker compose up en cours + curl “/version”

Question : Que gagne-t-on à décrire le lancement dans un fichier plutôt que dans une longue commande ?

Décrire le lancement dans docker-compose.yml permet de versionner toute la configuration dans Git et de démarrer l'application en une seule commande simple.

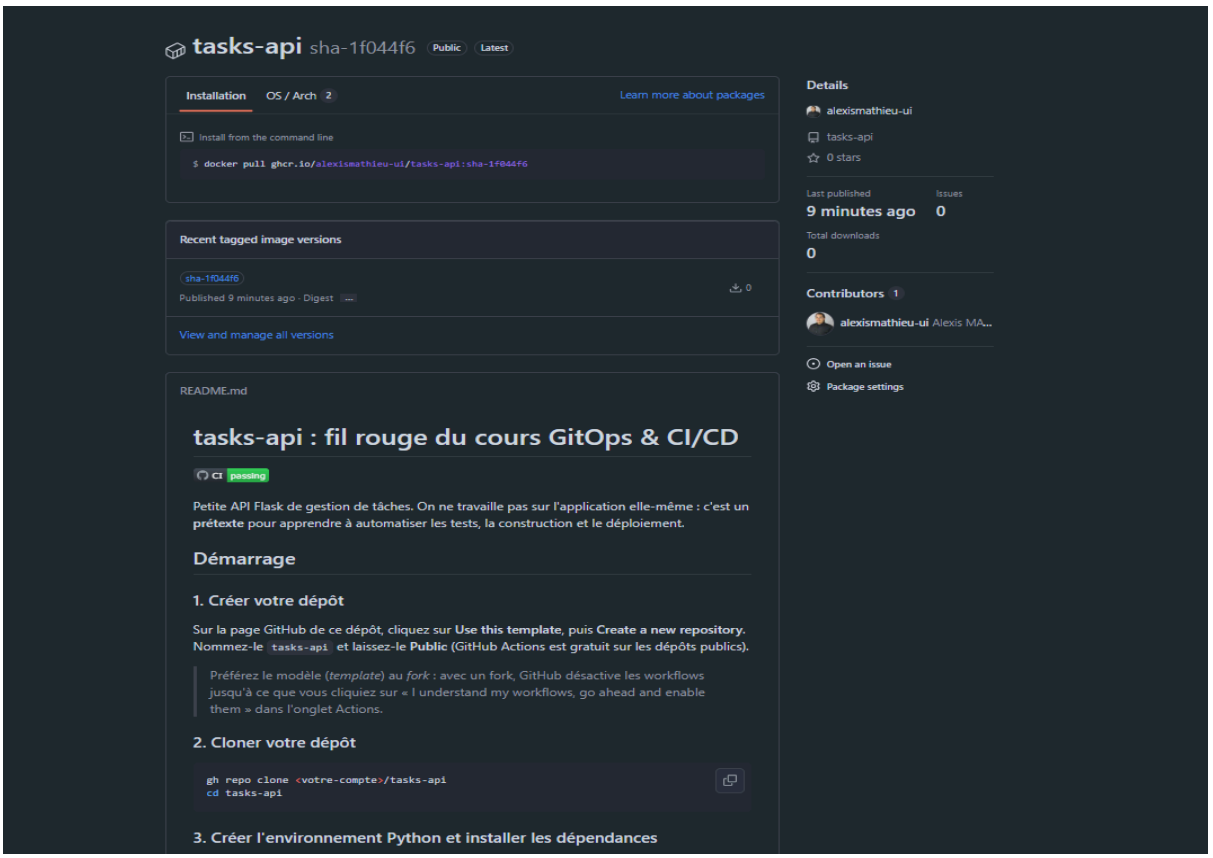
Partie 3

Screen 1 :



Légende : Run ‘test’ & ‘build’ vert

Screen 2:



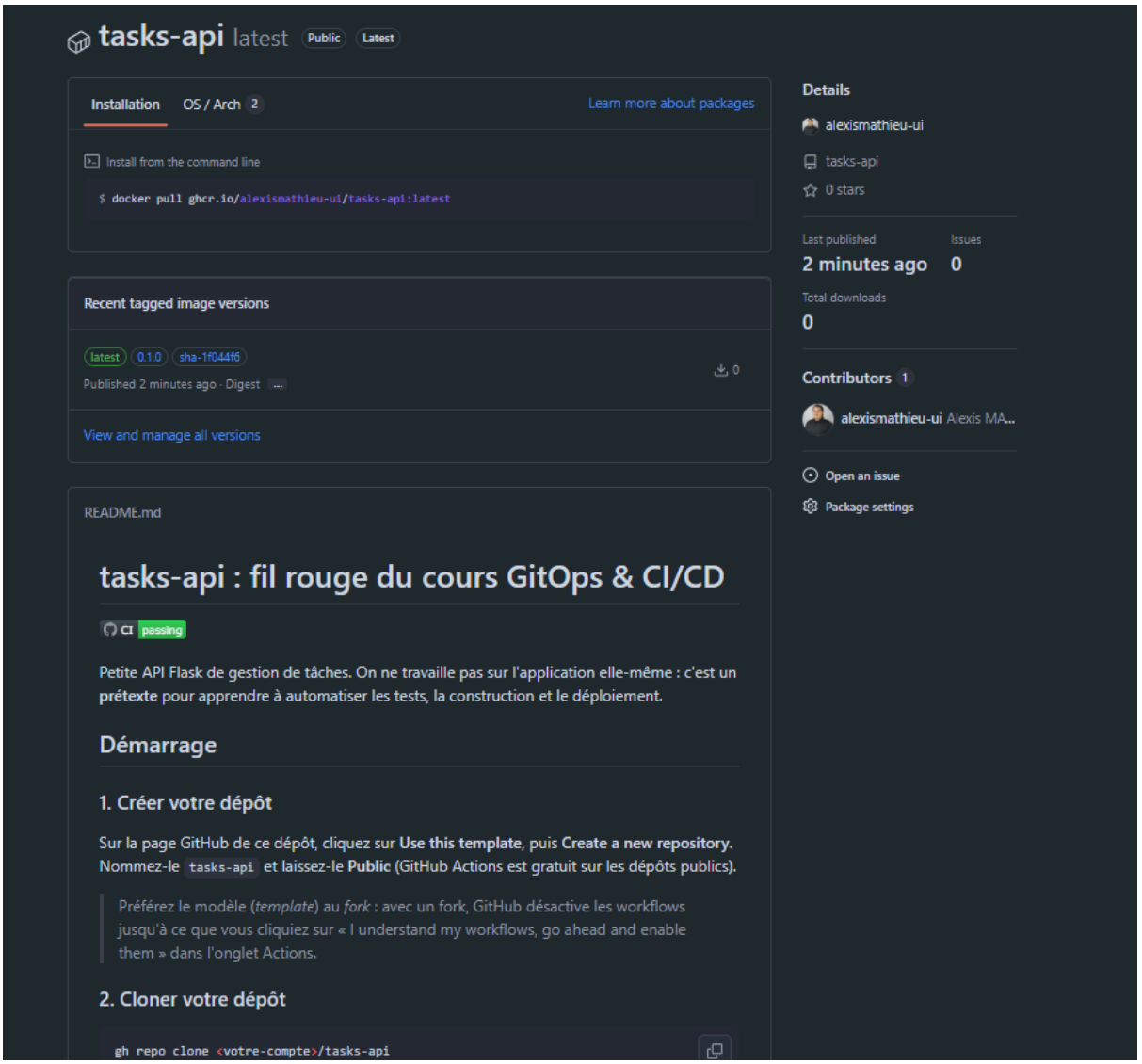
Légende : ‘task-api’ avec le tag ‘sha-…’

Question : Pourquoi `build` attend-il `test` ? Pourquoi éviter le tag `latest` en production ?

Le job build attend test pour garantir qu'on ne publie jamais sur le registre une image contenant un bug détecté par les tests.
On évite le tag latest en production car il change tout le temps, avec un tag précis, on assure la traçabilité et permet de revenir à la version précédente en cas de souci.

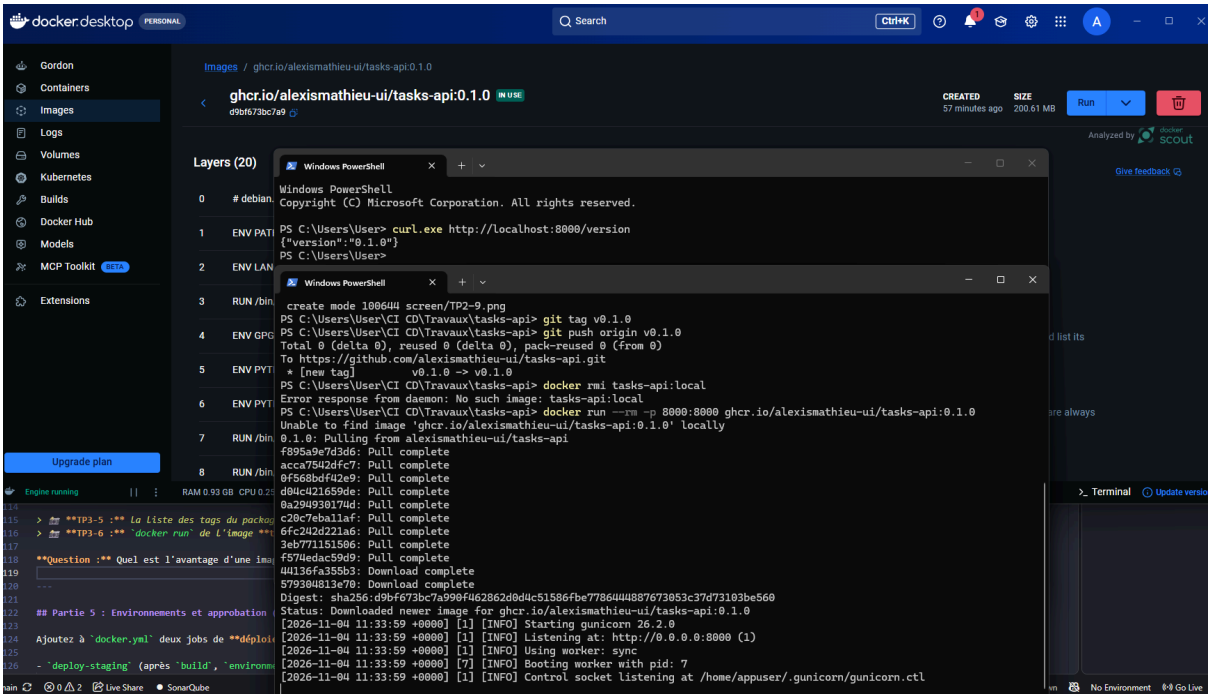
Partie 4

Screen 1 :



Légende : les tags ‘0.1.0’ et “sha-...”

Screen 2:



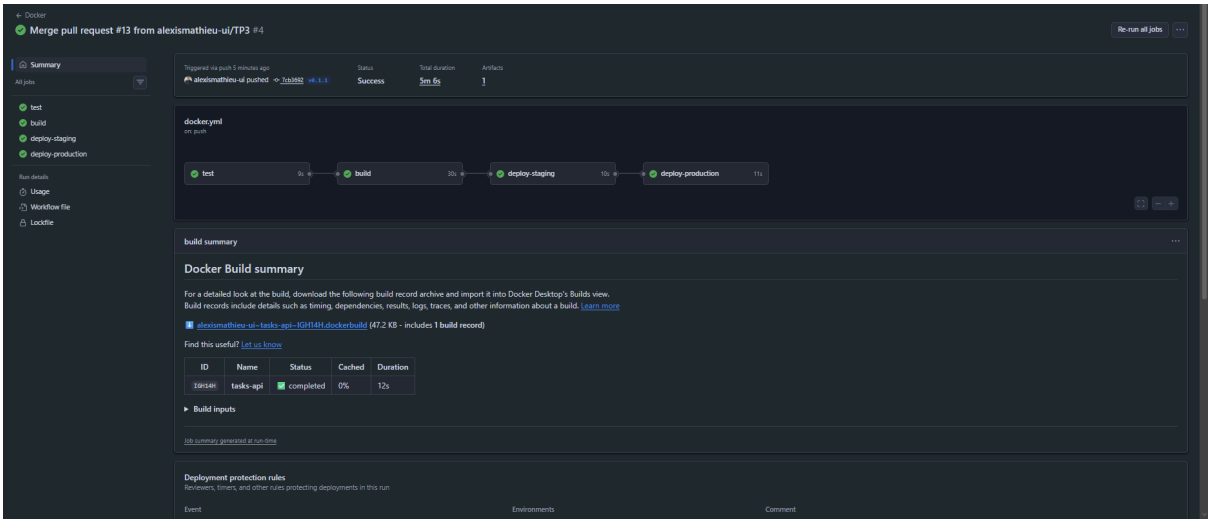
Légende : docker run + curl “/version”

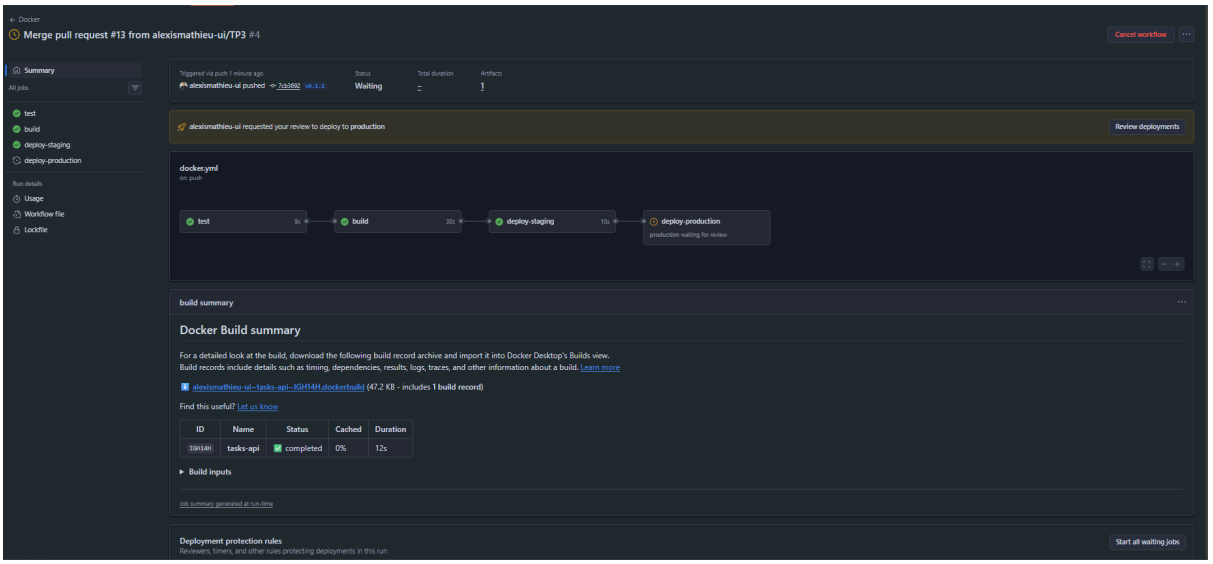
Question : Quel est l'avantage d'une image versionnée par rapport à « le code de ma machine » ?

Une image versionnée est validée par la CI et embarque exactement le même système et les mêmes bibliothèques, garantissant qu'elle tournera à l'identique sur n'importe quel serveur.

Partie 5

Screen 1 & 2:





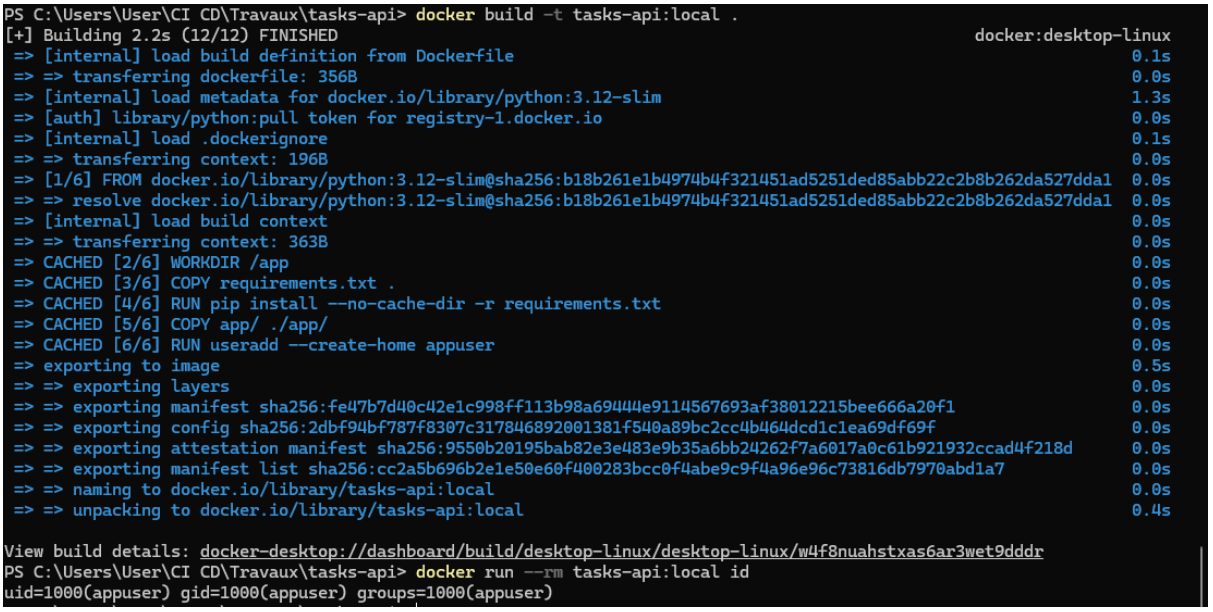
Légende : La run en attente (Screen 1) & run terminer (Screen 2)

Question : À quoi sert un test de fumée ? Pourquoi demander une approbation humaine avant la production ?

Le test de fumée démarre réellement le conteneur et appelle /health et /version pour vérifier que l'application se lance sans crasher.
L'approbation humaine avant la production permet de valider que tout fonctionne en staging et de maîtriser le moment exact de la mise en production.

Partie 6

Screen 1:



Légende : Sorti de la commande ‘id’

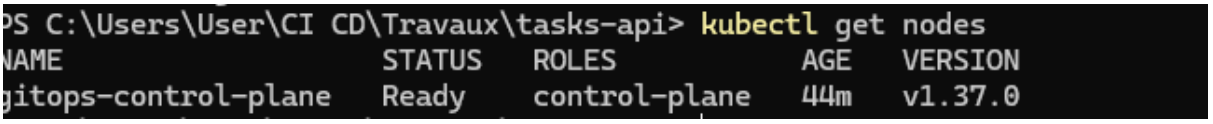
Question : Pourquoi ne pas exécuter une application en root dans un conteneur ?

On n'exécute pas une application en root dans un conteneur pour limiter les dégâts en cas de faille.

TP4 :

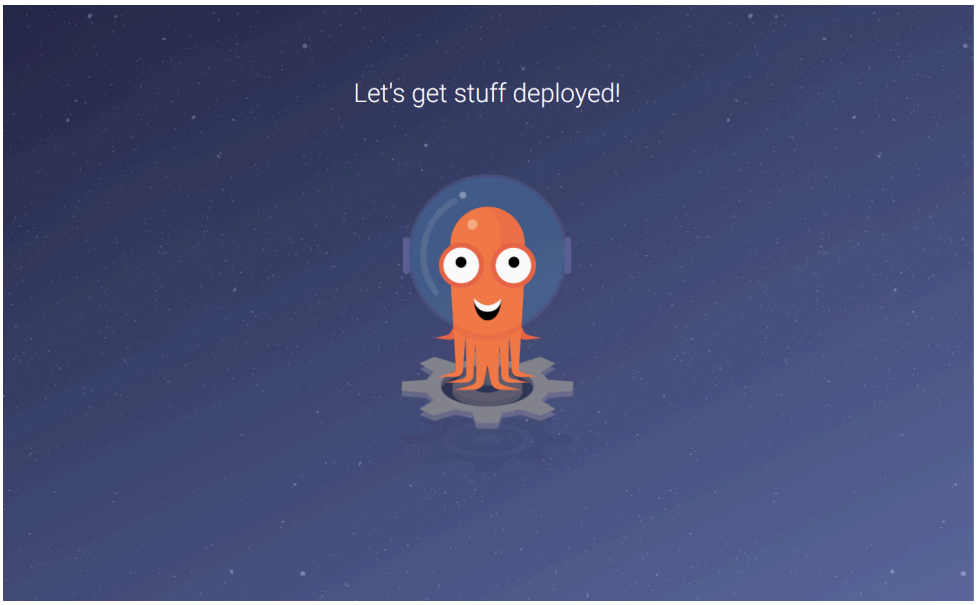
Partie 1

Screen 1:

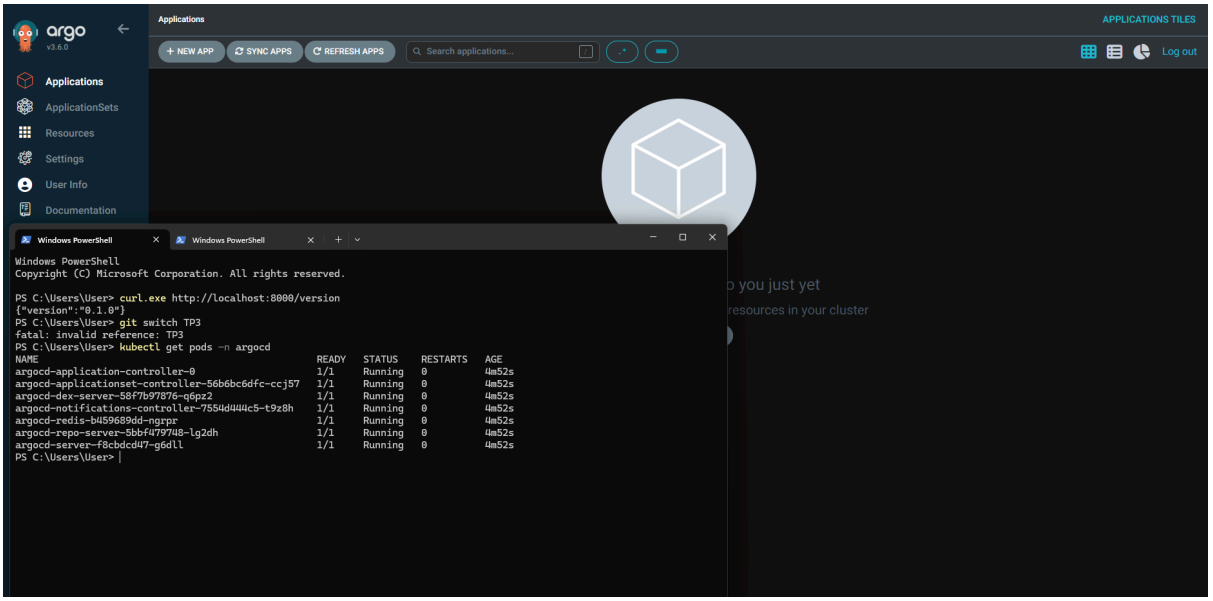


Légende : kubectl get nodes

Screen 2 & 3:







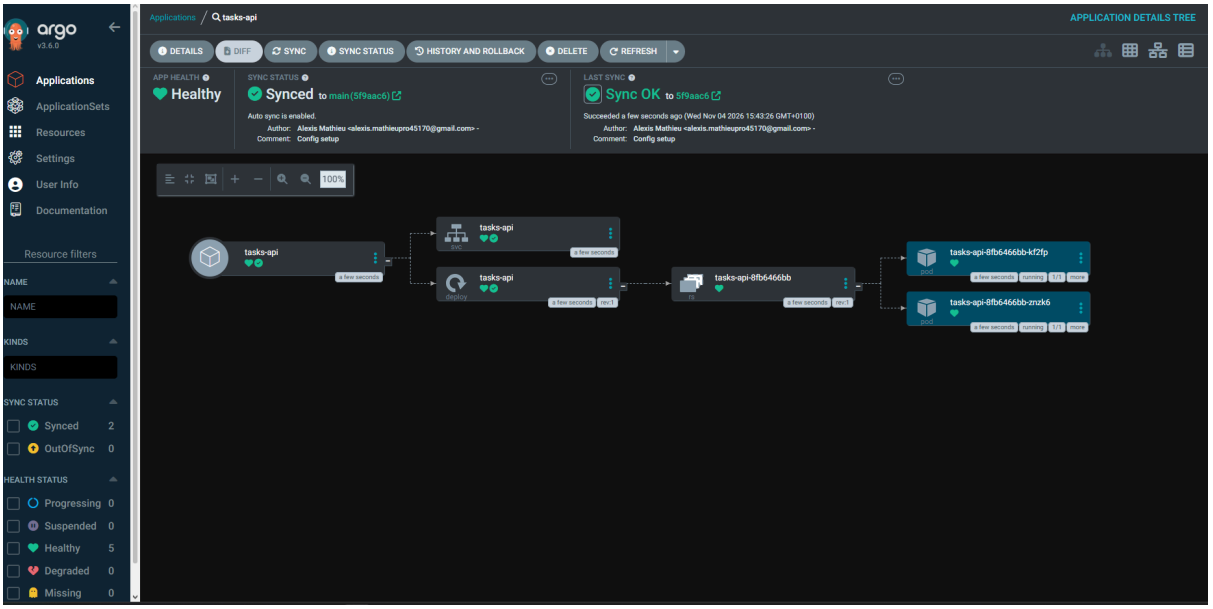
Légende : kubectl get pods -n argocd et Argo CD

Question : C'est quoi un pod ? Un deployment ? Pourquoi un `namespace` ?

Un pod est un conteneur éphémère qui fait tourner un conteneur de l'application, et un deployment s'assure de maintenir en vie le nombre voulu de pods.
Un namespace est un dossier logique qui isole les ressources dans le cluster.

Partie 2

Screen 1:



Légende : Application dans Argo CD, état Synced et Healthy, avec ses 2 pods

Screen 2:

```
PS C:\Users\User> curl.exe http://localhost:8081/version
{"version":"0.1.0"}
```

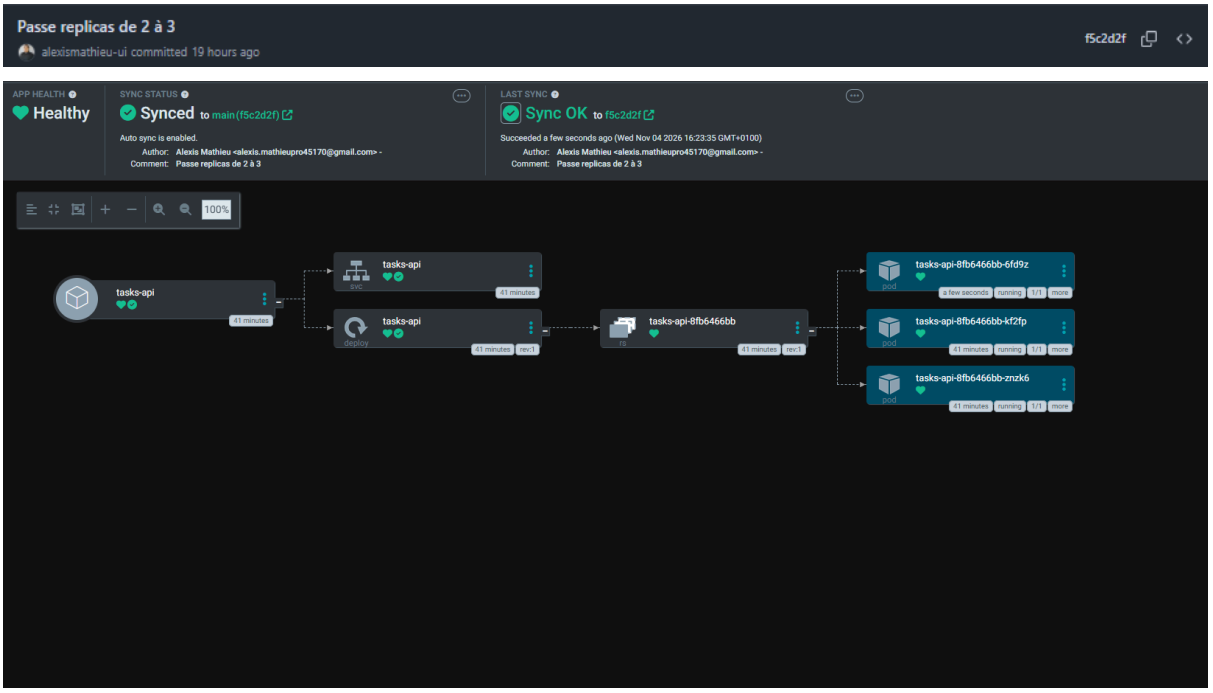
Légende : curl.exe /version

Question : Pourquoi sépare-t-on le dépôt de l'application et le dépôt de configuration ?

On sépare tasks-api et tasks-config car le pipeline de l'application ne déploie pas : il met à jour tasks-config , ce qui isole l'historique des déploiements et évite de relancer la CI applicative à chaque changement de configuration.

Partie 3

Screen 1 & 2:



Légende : Commit sur replicas: 3 et les 3 pods dans Argo CD

Screen 3:

```
PS C:\Users\User\CI CD\Travaux\tasks-api> kubectl scale deployment tasks-api -n tasks --replicas=5; kubectl get pods -n
tasks
deployment.apps/tasks-api scaled
NAME                         READY   STATUS              RESTARTS   AGE
tasks-api-8fb6466bb-6fd9z    1/1     Running             0          4m7s
tasks-api-8fb6466bb-g6hjl    0/1     ContainerCreating   0          0s
tasks-api-8fb6466bb-hfjft    0/1     ContainerCreating   0          0s
tasks-api-8fb6466bb-kf2fp    1/1     Running             0          44m
tasks-api-8fb6466bb-znzk6    1/1     Running             0          44m
PS C:\Users\User\CI CD\Travaux\tasks-api> kubectl get pods -n tasks
NAME                         READY   STATUS    RESTARTS   AGE
tasks-api-8fb6466bb-6fd9z    1/1     Running   0          4m25s
tasks-api-8fb6466bb-kf2fp    1/1     Running   0          44m
tasks-api-8fb6466bb-znzk6    1/1     Running   0          44m
```

Légende : Pods `kubectl scale` à 5 + correction automatique

Screen 4 & 5:





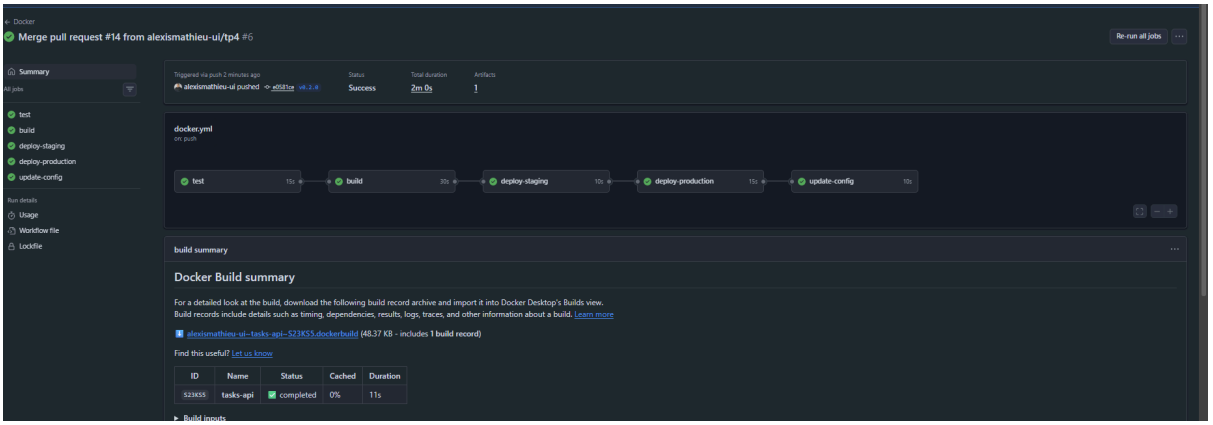
Légende : Commits de `tasks-config` avec `revert` + retour à 2 pods

Question : Que signifie `selfHeal` ? Pourquoi le retour arrière est-il simple en GitOps ?

‘selfHeal’ permet à Argo de détecter et d'annuler automatiquement toute modification manuelle sur le cluster qui s'écarte de l'état décrit dans Git.
Le retour arrière est simple en GitOps car un simple git revert restaure l'état précédent dans Git tout en gardant la trace de l'annulation dans l'historique.

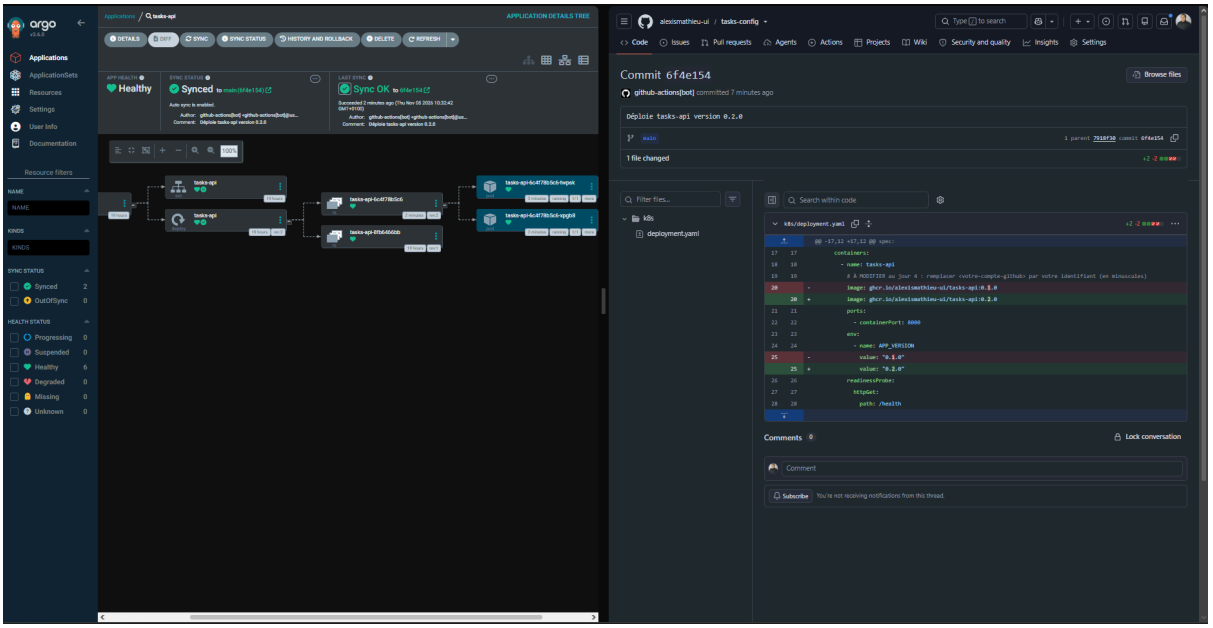
Partie 3

Screen 1:



Légende : Run tout vert

Screen 2:



Légende : Commit automatique dans tasks-config et Argo synchronisé sur 0.2.0

Screen 3:

```
PS C:\Users\User\CI CD\Travaux\tasks-api> curl.exe http://localhost:8081/version
{"version":"0.2.0"}
```

Légende : /version répond 0.2.0

Question : Résumez en 5 lignes le chemin d'un `git tag` jusqu'au pod. Qui pousse (push) et qui tire (pull) ? Quelle différence avec un déploiement CI/CD « classique » ?

Lorsqu'on pousse le tag v0.2.0, le pipeline lance les tests, publie l'image 0.2.0 sur GHCR, teste en staging, attend l'acceptation de production, puis commite le nouveau tag dans tasks-config. Ensuite, c'est Argo, qui pull ce changement depuis tasks-config et met à jour les pods : contrairement au CI/CD classique où le pipeline détient les clés d'administration du cluster pour lancer kubectl apply, en GitOps les clés ne sortent jamais du cluster et la dérive est corrigée en continu.